

Universidad de Valladolid

Práctica 2: Pruebas de carga, Locust y Prometheus

Grado en Ingeniería Informática
Evaluación de Sistemas Informáticos

Grupo 16

Índice

1. Introducción **2**
1.1. Descripción del sistema bajo prueba 2

2. Estructura del proyecto **2**

3. Instrumentación con Prometheus **3**
3.1. Conceptos previos 3
3.2. Instalación de la librería 3
3.3. Definición de métricas — metrics.py 4
3.4. Exposición del endpoint /metrics — app.py 5
3.5. Instrumentación de flujos OAuth2 — oauth2_routes.py 7
3.6. Verificación del sistema de métricas 8

4. Despliegue de Prometheus **8**
4.1. Instalación 8
4.2. Configuración 9
4.3. Arranque 9

5. Conexión con Grafana **9**
5.1. Arquitectura del sistema 9
5.2. Configuración del Data Source 10

1 Introducción

Esta práctica tiene como objetivo instrumentar un servidor de autenticación OAuth2 con métricas de Prometheus, visualizarlas en Grafana y posteriormente realizar pruebas de carga y estrés con Locust. El trabajo se divide en tres grandes bloques:

1. Instrumentación del backend con métricas Prometheus
2. Despliegue y configuración de Prometheus y Grafana
3. Pruebas de carga y estrés con Locust

1.1 Descripción del sistema bajo prueba

El backend proporcionado es un servidor de autenticación OAuth2 desarrollado en Python con Flask y SQLite, que implementa los tres flujos principales definidos en el RFC 6749 de la IETF:

- **Authorization Code**: flujo orientado a aplicaciones web donde un usuario humano se autentica mediante un formulario de login. El servidor emite un código temporal que posteriormente se canjea por un par de tokens.
- **Client Credentials**: flujo orientado a aplicaciones máquina a máquina donde no hay un usuario humano. La aplicación se autentica directamente con sus credenciales y obtiene un token de acceso.
- **Refresh Token**: flujo que permite renovar un token de acceso caducado sin necesidad de repetir el proceso de autenticación completo.

La aplicación se gestiona con `uv`, un gestor de paquetes moderno para Python, y se ejecuta en el puerto 5001.

2 Estructura del proyecto

El proyecto está compuesto por los siguientes ficheros:

Fichero	Descripción
`app.py`	Punto de entrada de la aplicación Flask. Registra blueprints, configura la base de datos y expone el endpoint `/metrics`.
`oauth2_routes.py`	Implementación de los endpoints OAuth2: `/authorize` y `/token`.
`admin_routes.py`	Endpoints de administración bajo `/admin/clients` para gestionar clientes OAuth2.
`models.py`	Capa de acceso a datos: clientes, códigos de autorización y tokens.
`database.py`	Gestión de la conexión SQLite y definición del esquema.
`metrics.py`	Definición centralizada de todas las métricas Prometheus (fichero creado en esta práctica).
`seed.py`	Script de inicialización que crea dos clientes de prueba en la base de datos.

Cuadro 1: Estructura de ficheros del proyecto

3 Instrumentación con Prometheus

3.1 Conceptos previos

Prometheus es una base de datos de series temporales de tipo *pull*, lo que significa que es la propia base de datos quien periódicamente consulta a las aplicaciones por sus métricas, en lugar de que las aplicaciones las envíen activamente.

Para que Prometheus pueda recoger las métricas, la aplicación debe exponer un endpoint `/metrics` con los datos en el formato de texto que Prometheus entiende. La librería `prometheus_client` para Python se encarga de gestionar este endpoint y formatear los datos automáticamente.

Prometheus define cuatro tipos de métricas, de las cuales utilizamos las dos más comunes:

- **Counter**: contador de incremento monotónico. Solo puede crecer. Se usa para contar eventos como peticiones recibidas, tokens emitidos o errores producidos. Por convención, los counters tienen el sufijo `_total`.
- **Gauge**: contador que puede crecer y decrecer. Se usa para valores que fluctúan como memoria usada o conexiones activas.

3.2 Instalación de la librería

La librería oficial de Prometheus para Python se instala con:

```
uv add prometheus-client
```

El comando `uv add` instala la librería y actualiza automáticamente `pyproject.toml` y `uv.lock` para registrar la dependencia.

3.3 Definición de métricas — metrics.py

Se crea un fichero `metrics.py` centralizado con todos los contadores, evitando así importaciones circulares entre módulos:

```
from prometheus_client import Counter

# Flujos de autenticacion
auth_flows_started = Counter(
"oauth2_auth_flows_started_total",
"Flujos de autenticacion iniciados",
["grant_type"]
)
auth_flows_success = Counter(
"oauth2_auth_flows_success_total",
"Flujos de autenticacion completados con exito",
["grant_type"]
)
auth_flows_failed = Counter(
"oauth2_auth_flows_failed_total",
"Flujos de autenticacion fallidos",
["grant_type"]
)

# Tokens emitidos (general)
tokens_issued = Counter(
"oauth2_tokens_issued_total",
"Tokens emitidos",
["token_type", "grant_type"]
)

# Tokens emitidos (separados por tipo)
access_tokens_issued = Counter(
"oauth2_access_tokens_issued_total",
"Access tokens emitidos",
["grant_type"]
)
refresh_tokens_issued = Counter(
"oauth2_refresh_tokens_issued_total",
"Refresh tokens emitidos",
["grant_type"]
)

# Llamadas HTTP (general)
http_requests = Counter(
"oauth2_http_requests_total",
"Numero de llamadas HTTP",
["method", "endpoint", "status_code"]
```

```
        )

        # Llamadas HTTP (separadas por endpoint)
        http_requests_token = Counter(
        "oauth2_http_requests_token_total",
        "Llamadas al endpoint /token",
        ["method", "status_code"]
        )
        http_requests_authorize = Counter(
        "oauth2_http_requests_authorize_total",
        "Llamadas al endpoint /authorize",
        ["method", "status_code"]
        )
        http_requests_admin = Counter(
        "oauth2_http_requests_admin_total",
        "Llamadas a los endpoints /admin",
        ["method", "status_code"]
        )
```

Listing 1: metrics.py

Las **etiquetas** (*labels*) permiten que un mismo counter almacene valores independientes para cada combinación de etiquetas. Por ejemplo, `auth_flows_started` con la etiqueta `grant_type` almacena un contador separado para cada tipo de flujo:

```
oauth2_auth_flows_started_total{grant_type="
   client_credentials"} 10.0
oauth2_auth_flows_started_total{grant_type="
   authorization_code"} 3.0
oauth2_auth_flows_started_total{grant_type="
   refresh_token"} 5.0
```

3.4 Exposición del endpoint /metrics — app.py

Se modifica `app.py` para exponer el endpoint `/metrics` y registrar automáticamente todas las peticiones HTTP mediante decoradores Flask:

```
        from flask import Flask, request, g
        from prometheus_client import make_wsgi_app
        from werkzeug.middleware.dispatcher import
           DispatcherMiddleware
        from werkzeug.serving import run_simple
        import time

        from metrics import (
        http_requests,
        http_requests_token,
        http_requests_admin,
        http_requests_authorize
        )

```

```
        def create_app() -> Flask:
        # ... configuracion existente ...

        @app.before_request
        def before_request():
        # Guardamos el instante de llegada de la peticion
        g.start_time = time.time()

        @app.after_request
        def after_request(response):
        endpoint = request.path

        # Counter general de peticiones HTTP
        http_requests.labels(
        method=request.method,
        endpoint=endpoint,
        status_code=str(response.status_code)
        ).inc()

        # Counters especificos por endpoint
        if "/token" in endpoint:
        http_requests_token.labels(
        method=request.method,
        status_code=str(response.status_code)
        ).inc()
        elif "/authorize" in endpoint:
        http_requests_authorize.labels(
        method=request.method,
        status_code=str(response.status_code)
        ).inc()
        elif "/admin" in endpoint:
        http_requests_admin.labels(
        method=request.method,
        status_code=str(response.status_code)
        ).inc()

        return response

        return app

        if __name__ == "__main__":
        flask_app = create_app()

        # Combinamos Flask con el endpoint /metrics de
           Prometheus
        application = DispatcherMiddleware(flask_app, {
            "/metrics": make_wsgi_app()
        })

        run_simple("0.0.0.0", 5001, application,
        use_reloader=True, use_debugger=True)
```

Listing 2: Fragmentos añadidos a app.py

Los **decoradores** `@before_request` y `@after_request` son hooks de Flask que se ejecutan automáticamente antes y después de cada petición HTTP respectivamente. Esto permite registrar todas las peticiones en un único lugar sin modificar cada ruta individualmente, siguiendo el principio DRY (*Don't Repeat Yourself*).

El `DispatcherMiddleware` combina la aplicación Flask con la mini aplicación de Prometheus que genera el endpoint `/metrics`, de modo que todo el tráfico a `/metrics` lo gestiona Prometheus y el resto lo gestiona Flask.

3.5 Instrumentación de flujos OAuth2 — oauth2_routes.py

Se modifica `oauth2_routes.py` para incrementar los contadores en cada uno de los tres flujos de autenticación. La lógica es la misma para los tres: se incrementa `started` al entrar en la función, `failed` en cada punto de fallo y `success` junto con los contadores de tokens al completarse con éxito.

A continuación se muestra el flujo `client_credentials` como ejemplo:

```
        def _handle_client_credentials():
        auth_flows_started.labels(
        grant_type="client_credentials").inc()

        client_id, err = _authenticate_client()
        if err:
        auth_flows_failed.labels(
        grant_type="client_credentials").inc()
        return _error_response("invalid_client", err, 401)

        client = get_client(client_id)
        if client is None or \
        "client_credentials" not in client["grant_types"]:
        auth_flows_failed.labels(
        grant_type="client_credentials").inc()
        return _error_response("unauthorized_client",
        "Client is not authorized for client_credentials")

        scope = request.form.get("scope", "")
        token_response = issue_token_pair(
        client_id=client_id,
        scope=scope,
        user_id=None,
        include_refresh=False,
        )

        auth_flows_success.labels(
        grant_type="client_credentials").inc()
        tokens_issued.labels(
        token_type="access",
```

```
31          grant_type="client_credentials").inc()
32          access_tokens_issued.labels(
33          grant_type="client_credentials").inc()
34          # client_credentials no emite refresh token (RFC 6749)
35
36          return jsonify(token_response)
```

Listing 3: Fragmento de _handle_client_credentials()

Los flujos `authorization_code` y `refresh_token` siguen la misma lógica pero con más puntos de fallo y emiten además un `refresh_token`, por lo que incrementan cinco contadores en lugar de tres al completarse con éxito.

3.6 Verificación del sistema de métricas

Tras arrancar la aplicación y realizar dos peticiones de prueba (una correcta y una con credenciales incorrectas), se verifica el correcto funcionamiento consultando el endpoint `/metrics`:

```
curl http://localhost:5001/metrics | grep
   oauth2
```

Los resultados obtenidos son los esperados:

```
oauth2_auth_flows_started_total{grant_type="
   client_credentials"} 2.0
oauth2_auth_flows_success_total{grant_type="
   client_credentials"} 1.0
oauth2_auth_flows_failed_total{grant_type="
   client_credentials"} 1.0
oauth2_tokens_issued_total{grant_type="
   client_credentials",
        token_type="access"} 1.0
oauth2_http_requests_total{endpoint="/token",
        method="POST",status_code="200"} 1.0
oauth2_http_requests_total{endpoint="/token",
        method="POST",status_code="401"} 1.0
```

Los contadores cuadran perfectamente: 2 flujos iniciados, 1 exitoso (petición correcta) y 1 fallido (credenciales incorrectas). Solo se emitió 1 token de acceso, correspondiente al flujo exitoso. No se emitieron refresh tokens porque el flujo `client_credentials` no los emite según el RFC 6749.

4 Despliegue de Prometheus

4.1 Instalación

Se descarga el binario oficial de Prometheus desde GitHub y se extrae:

```
wget https://github.com/prometheus/prometheus/
   releases/download/\
v3.3.1/prometheus-3.3.1.linux-amd64.tar.gz
tar -xvf prometheus-3.3.1.linux-amd64.tar.gz
cd prometheus-3.3.1.linux-amd64
```

4.2 Configuración

Se edita el fichero `prometheus.yml` para añadir un nuevo job que apunte a nuestra aplicación en el puerto 5001:

```
global:
# Intervalo de busqueda de metricas en
   localhost:5001
scrape_interval: 15s
# Intervalo de evaluacion de reglas de alertas
# No se usa en esta practica
evaluation_interval: 15s

scrape_configs:
- job_name: "prometheus"
static_configs:
- targets: ["localhost:9090"]

# Job que apunta a nuestra aplicacion OAuth2
- job_name: "oauth2"
static_configs:
- targets: ["localhost:5001"]
```

Listing 4: prometheus.yml

El parámetro `scrape_interval` define cada cuánto Prometheus va a buscar métricas a la aplicación. El parámetro `evaluation_interval` define cada cuánto evalúa las reglas de alertas, que no se utilizan en esta práctica.

4.3 Arranque

```
./prometheus --config.file=prometheus.yml
```

Prometheus queda corriendo en el puerto 9090 y empieza a recoger métricas de nuestra aplicación cada 15 segundos.

5 Conexión con Grafana

5.1 Arquitectura del sistema

Una vez desplegados los tres componentes, la arquitectura queda así:

Componente	Puerto	Función
App OAuth2	5001	Genera métricas en `/metrics`
Prometheus	9090	Recoge y almacena las métricas cada 15s
Grafana	3000	Visualiza las métricas en dashboards

Cuadro 2: Arquitectura del sistema de monitorización

El flujo de datos es el siguiente:

`App OAuth2 (5001)` → `Prometheus (9090)` → `Grafana (3000)`

5.2 Configuración del Data Source

Se accede a Grafana en `http://localhost:3000` y se añade Prometheus como fuente de datos en *Connections >Data Sources >Add data source*. Se configura la URL `http://localhost:9090` y se verifica la conexión con *Save & Test*, obteniendo el mensaje:

"Successfully queried the Prometheus API."

Esto confirma que Grafana puede acceder a los datos almacenados en Prometheus y está listo para visualizarlos en dashboards mediante consultas PromQL.